



Autenticação Google em Express.js e NestJS

OAuth 2.0 • OpenID Connect • Passport • JWT

O que é Autenticação Google?

SEM GOOGLE



Email + Senha



Nossa API



 JWT

COM GOOGLE



 Google Login



Nossa API



 JWT

O que acontece por trás?

OAuth 2.0

Fluxo padrão de autorização externa.

Passport

Integração da autenticação com o backend.

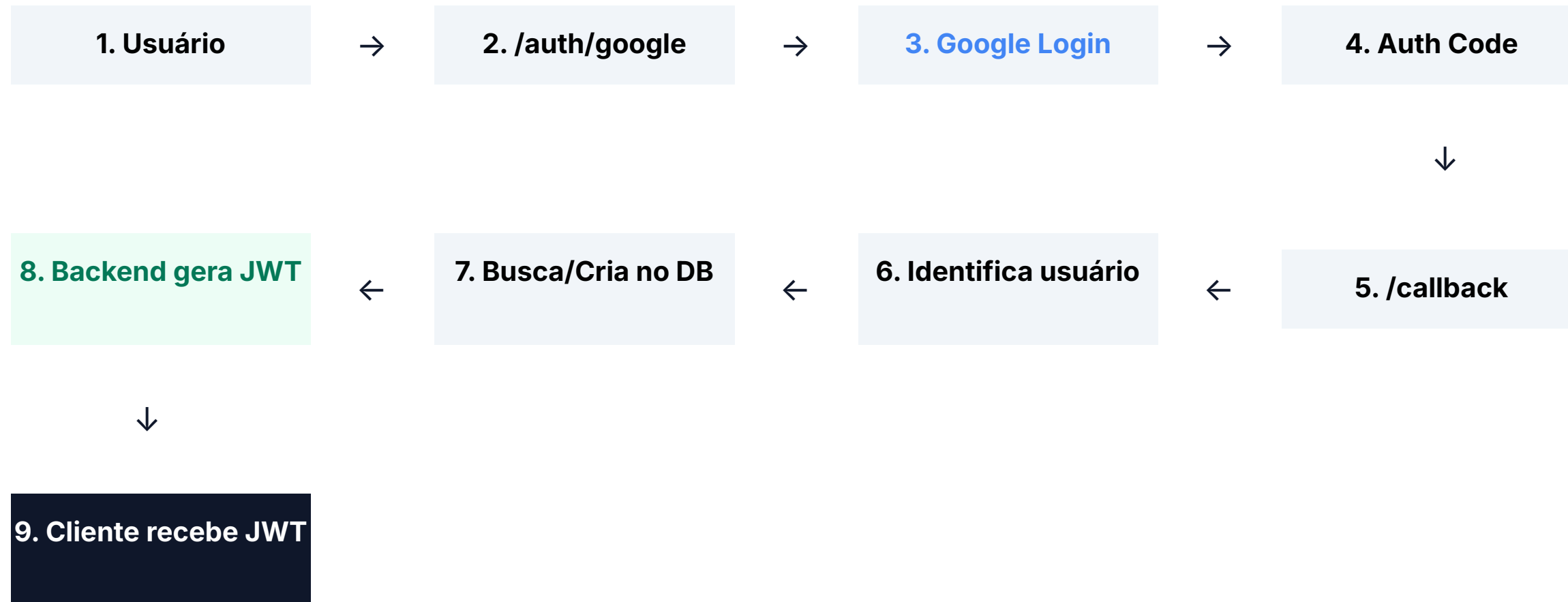
OpenID Connect

Camada de identidade sobre OAuth 2.0.

JWT

Token interno gerado pela nossa aplicação após login.

O Fluxo Completo de Autenticação



Implementação com Express.js

Route



Passport Middleware



Google Strategy



Controller / Handler



Gera JWT

```
// Inicia o login e redireciona ao Google router.get(
"/google", passport.authenticate("google", { scope: ["openid",
"profile", "email"], session: false }) );
```

Utiliza Passport

Não utiliza sessão

Fluxo baseado em JWT

Express: Callback e Geração do JWT

Google Autentica



/google/callback



req.user (Passport)



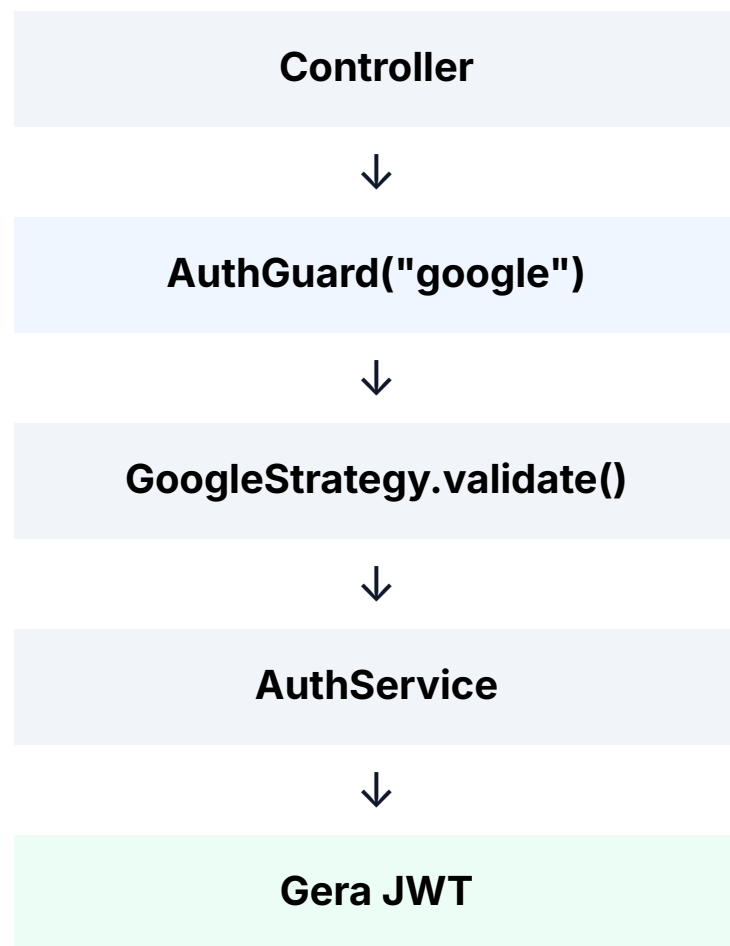
Aplicação gera JWT



Resposta JSON

```
router.get( "/google/callback", passport.authenticate("google", {  
  session: false }), (req, res) => { const token = jwt.sign( { sub:  
  req.user.id, email: req.user.email }, process.env.JWT_SECRET, {  
  expiresIn: "1h" } ); res.json({ token }); } );
```

Implementação com NestJS



```
// Rota inicial no Controller @Get("google")
@UseGuards(AuthGuard("google")) googleLogin() { //
  Redirecionamento automático }
```

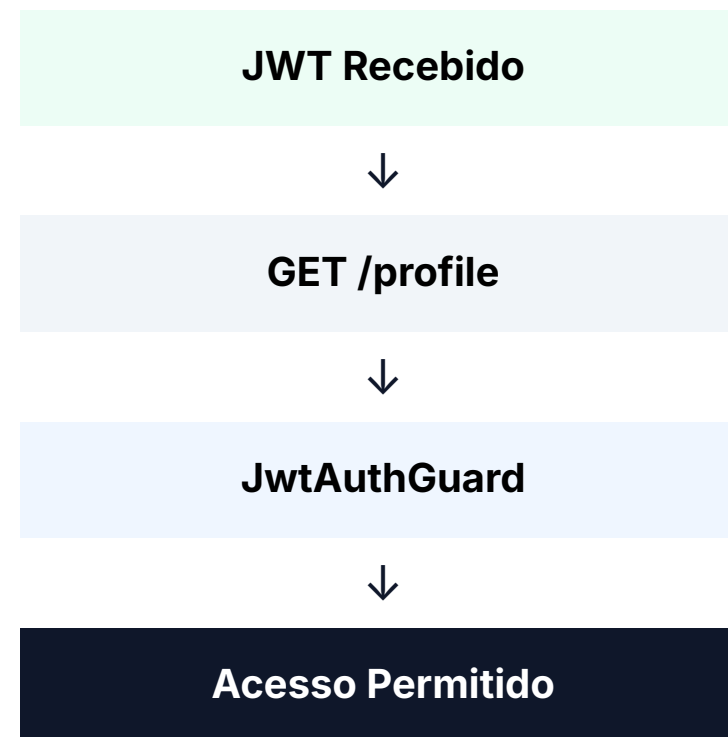
A Grande Diferença

- **Express:** Usa funções Middleware.
- **NestJS:** Usa Guards e Decorators.
- O conceito e o fluxo OAuth são exatamente os mesmos.

NestJS: JWT e Rotas Protegidas

```
// Protegendo uma rota com o JWT gerado @Get("profile")  
@UseGuards(JwtAuthGuard) getProfile(@Req() req) { return  
  req.user; }
```

Google autentica a identidade. O JWT protege as próximas requisições na nossa API.



Comparação de Implementação

Conceito	Express.js	NestJS
Integração do Passport	Middleware	AuthGuard
Invocação da rota	<code>passport.authenticate()</code>	<code>@UseGuards(AuthGuard('google'))</code>
Lógica de Validação	Google Strategy Callback	Google Strategy validate()
Estrutura do Código	Configuração mais manual	Arquitetura modular rígida

O fluxo de autenticação é praticamente o mesmo; a organização do código muda.

Demonstração Prática

1. OBTENÇÃO DO TOKEN

GET /auth/google



Login no Google



GET /auth/google/callback



Retorna JWT (eyJhb...)

2. ACESSO À ROTA PROTEGIDA

GET /auth/profile

Headers:

Authorization: Bearer

Resposta (200 OK):

```
{ "id": 1, "name": "Usuário", "email":  
  "usuario@gmail.com" }
```

O que aprendemos?

1. Google

Autentica a identidade do usuário de forma segura.

2. Passport

Integra e padroniza o fluxo OAuth no backend.

3. JWT

Mantém a sessão ativa dentro da nossa própria API.

4. Express x Nest

Conceito idêntico, mecanismos de framework diferentes.

Google → Autenticação → Backend → **JWT** → Rotas Protegidas